

17. Bölüm

Veri Tabanı İşlemleri

17.1 Veri Tabanı Nedir?

Veri tabanları (**database**), tuttukları veriler hakkında bilgi sahibi olan programlardır. Veri tabanları, yazdığımız uygulamadan bağımsız olarak verilerimizi sistematik olarak saklar, erişilebilir kılar ve yönetmemizi sağlar. Veri tabanı kullanmak bize veri tutarlılığı, veri paylaşımı, veri güvenliği ve veri analitiği için bir altyapı sağlar. Veri tabanı;

- ♦ Kullanıcılardan gelen sorgu taleplerini analiz eder,
- ♦ Analiz sonrasında varsa mevcut veri üzerinde değişiklik yapar veya istenen veriyi belirler
- ♦ Sorgu sonucunu kullanıcının erişebileceği formda sunar.

Veri tabanında sorgular, **SQL (Structured Query Language)** dili kullanan **ilişkisel veri tabanında** veya **NoSQL** yaklaşımlarıyla **ilişkisel olmayan veri tabanlarında** yapılmaktadır. NoSQL bir dilden ziyade bir veri modelleme yaklaşımıdır (Document, Key-Value, Graph benzeri). Veri tabanları, **veri tabanı yönetim sistemleri** (VTYS) aracılığıyla güvenli ve ölçeklenebilir şekilde yönetilebilmektedir. Veri tabanı yönetim sistemi verinin fiziksel veya mantıksal alanda bulunduğunu belirler ve gelen sorguları bu kapsamda ayrıştırmaktadır. Sorguları hızlandırmak için indeksler oluşturulup veriye hızlıca erişilmesi sağlanmaktadır.

Bir uygulama geliştirmeye başladığınızda yapacağınız ilk tercihlerden biri **SQL** veya **NoSQL** veritabanı kullanıp kullanmayacağınızdır. Verileri düzenlemek çok zor bir iştir ve düzenlediğimizde verilerimizi çeşitli türlere göre kategorize ederiz;

1. **PostgreSQL, MySQL, Microsoft SQL Server, Oracle Database** veya **SQLite** gibi geleneksel bir veri tabanları şema (**schema**) kullanır ve bu veri tabanlarında **saklanmış verileri düzenlemek** daha kolaydır.
2. **MongoDB, Redis, CouchDB, Cassandra, Elasticsearch, Couchbase** ve **LiteDB** gibi NoSQL veri tabanları SQL veri tabanları kadar **veri tutarlılığı sağlamaz ve saklanmış verileri düzenlemek çok daha zordur**. Daha çok iz kayıtları (**log**), belge ve resim saklamak için kullanılırlar.

SQL ve NoSQL arasındaki temel fark, verilerin nasıl depolandığı ve nasıl dışarıya aktarıldığıdır.

Proje İçin Veri Tabanının Erişilebilir Olması

Bir **veri tabanı projesini yazabilmeniz için ilk önce** ilgili veri tabanına karar verip projenizin çalıştığı bilgisayara ya da projenizle aynı ağda ve **erişilebilir olan bir bilgisayara kurmanız gerekir**. **Erişilebilir bir veri tabanınız varsa o veri tabanına ilişkin bileşenleri projemize dahil ederek** veri tabanına bağlanıp gerekli veri işlemlerini gerçekleştirebiliriz.

Visual Studio ile **3.11.2** başlığındaki gibi oluşturulmuş projelere veritabanlarını kullanacak bileşenlerini dahil etmek için **Tools→NuGet Package Manager→Package Manager Console** üzerinden komut vererek kullanacağımız veritabanına ilişkin bileşenleri projemize dahil ederiz;

```
PM>Install-Package Microsoft.Data.Sqlite
PM>Install-Package Microsoft.Data.SqlClient
PM>Install-Package LiteDb
```

.NET Core CLI ile **3.11.1** başlığındaki gibi oluşturulmuş projelere veritabanlarını kullanacak bileşenlerini dahil etmek için proje klasöründe aşağıdaki komutlar verilir;

```
$dotnet add package Microsoft.Data.Sqlite
$dotnet add package Microsoft.Data.SqlClient
$dotnet add package LiteDb
```

Çalışılacak Veritabanı Bileşenlerinin Projeye Dahil Edilmesi

Veri tabanını kullanacak **her projeye** yukarıda verilen paketlerden ilgili olanı **dahil edilmeli/kurulmalıdır!**

17.2 Bağlantı Metni

Projemizde kullanacağımız veri tabanları uygulamamızdan ayrı bir programdır. Bu veri tabanlarına bağlanmamızı sağlayan metinleri (**string**) değişkenleri olarak tanımlarız. Bu değişkenlere **bağlantı metni** (**connection string**) adı veririz. Bu değişken genellikle veri tabanı yönetim sistemine iletilen kullanıcı adı, şifre ve şema bilgileri içerir.

```
string sqliteConnectionString="Data Source=hello.db"; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde oluşur.
var sqliteConnection = new SqliteConnection(sqliteConnectionString);
string nosqlConnectionString=@"MyData.db"; //Çalıştırılabilir .NET uygulamasının olduğu klasörde
→ oluşur.
var nosqlConnection = new LiteDatabase(nosqlConnectionString);
```

Küçük ve yerel bilgisayarınızda çalışabilecek veri tabanlarının bağlantı metinleri kısa ve öz olur. Ancak kurumsal ve ölçeklenebilir Veri tabanlarının bağlantı dizgisi daha karmaşık olabilir. Bu gibi bağlantı metinlerini oluşturmak için **DbConnectionStringBuilder**, **SqlConnectionStringBuilder** gibi yardımcı sınıflar kullanırız;

```
DbConnectionStringBuilder builder1 = new DbConnectionStringBuilder();
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\")); //Proje klasörü
var db = Path.Combine(projeKlasörü, "MyDb.mdb");//Access DataBase
builder1.ConnectionString = $"Data Source={db}";
builder1.Add("Provider", "Microsoft.Jet.Oledb.4.0");
builder1.Add("Jet OLEDB:Database Locking Mode", 1);
OleDbConnection oledbConnect = new OleDbConnection(builder1.ConnectionString);

var builder2 = new SqlConnectionStringBuilder {
    DataSource = "(local)",
    UserID = "sa",
    Password = "123456*",
    InitialCatalog = "CSharpDB"
};
var SqlConnection = new SqlConnection(builder2.ConnectionString);
var SqlConnection2 = new SqlConnection("server=(local);initial catalog=CSharpDB");
```

Veri kaynakları genellikle hem mali hem de tükettikleri kaynak açısından pahalı sistemlerdir. **Pahalı olan bu kaynaklar başka yazılımlar ile ortak kullanılırlar. Bu nedenle veri kaynaklarına olan bağlantılarımızı sınırlı tutmalıyız.** Bu tür kaynakları etkin kullanmak için **using** saklı kelimesi kullanılır. **using** kelimesi veri kaynağı gibi değerli kaynaklara ilişkin işlem yapılırken bir hata meydana geldiğinde, bağlantıya ilişkin tüm nesneleri **Dispose()** yöntemi ile bellekten siler yani **Dispose()** yönteminin çağrılmasını garanti eder.

17.3 İlişkisel Veri Tabanı İşlemleri

Bir veri tabanını kullanabilmek için ilk önce bir bağlantı (**connection**) kurmak gerekir. Sonrasında veri tabanı üzerinde işlem yapacak SQL komutu (**command**) nesnesi hazırlanır ve hazırlanan komut icra (**execute**) edilir. SQL komutuna ilişkin **CommandText** özelliğine noktalı virgül karakteriyle ayırarak veritabanı üzerinde birden fazla sorgu çalıştırabiliriz. Bu bölümde verilen örneklerde SQL sorgusu bildiğiniz kabul edilmiştir.

§17.3.1 Veri Tabanında Tablolar Oluşturma

Aşağıdaki örnek kod ile SQLite veri tabanında **Oğrenci** ve **Ders** adında iki tablo oluşturulmuştur. Oluşturulan tablolara 6 adet öğrenci ve 2 adet ders kaydı atılmıştır.

```
using Microsoft.Data.Sqlite; //Nuget Paketini Kur!

var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\")); //Proje klasörü
var db = Path.Combine(projeKlasörü, "CSharp.sqlite");
var connectionString = $"Data Source={db}";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
```

```

var command = connection.CreateCommand();
command.CommandText = @"
    DROP TABLE IF EXISTS Ogrenci;
    CREATE TABLE Ogrenci (
        OgrenciId INTEGER PRIMARY KEY AUTOINCREMENT,
        Adi text NOT NULL,
        Soyadi text NOT NULL
    );
    DROP TABLE IF EXISTS Ders;
    CREATE TABLE Ders(
        DersId INTEGER PRIMARY KEY AUTOINCREMENT,
        DersAdi NVARCHAR(50)
    );
";
int rowsAffected = command.ExecuteNonQuery(); //Veri getirmeyecek bir sorgu!
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi."); //0 kadar kayıt etkilendi.
command.CommandText = @"
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ali', 'Veli');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ayşe', 'Fatma');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Mehmet', 'Can');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Zeynep', 'Yılmaz');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ahmet', 'Demir');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Fatma', 'Kaya');
    INSERT INTO Ders (DersAdi) VALUES ('Matematik');
    INSERT INTO Ders (DersAdi) VALUES ('Fizik');
";
rowsAffected = command.ExecuteNonQuery();
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi."); //8 kadar kayıt etkilendi.
connection.Close();
}

```

Visual Studio açık iken **Extensions** → **Manage Extensions...** tıklanarak arama ekranında *SQLite/SQL Compact Toolbox* eklentisini arayıp kurabilirsiniz. Kurulum sonrasında SQLite veri tabanını Şekil 17.1'deki gibi açıp sorgulayabiliriz ¹.

§17.3.2 Veri Tabanından Veri Okuma

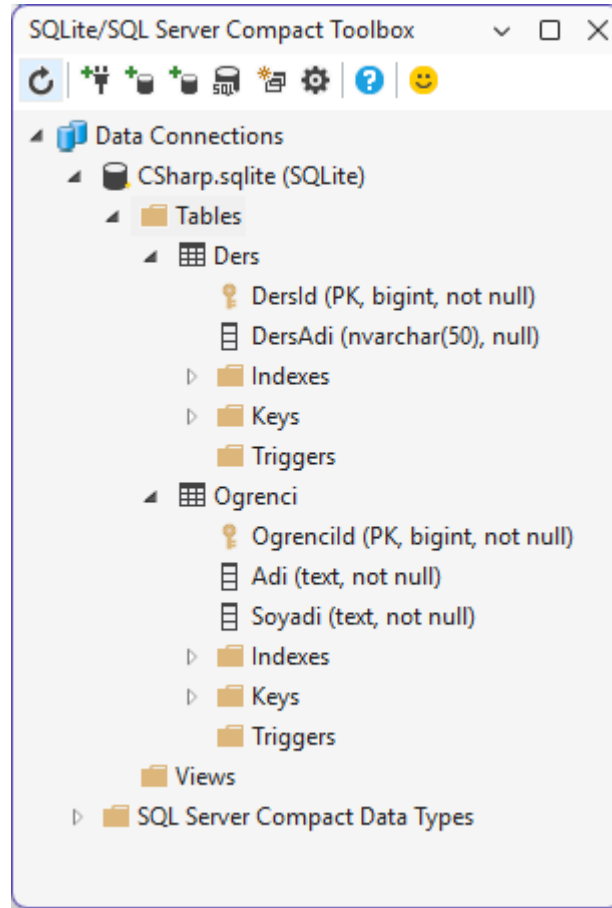
Veri tabanına eklenmiş kayıtları da sorgulatıp öğrenebiliriz;

```

using Microsoft.Data.Sqlite; //Nuguet Paketini Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\")); //Proje klasörü
var db = Path.Combine(projeKlasörü, "CSharp.sqlite");
var connectionString = $"Data Source={db}";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText = @"SELECT OgrenciId, Adi, Soyadi FROM Ogrenci;";
    using (SqliteDataReader reader = command.ExecuteReader()) //Veri getirecek bir sorgu!
    {
        while (reader.Read())
        {
            var öğrenciId = reader.GetString(0);
            var adı = reader.GetString(1);
            var soyadı = reader.GetString(2);
            Console.WriteLine($"({öğrenciId}) {adı} {soyadı}");
        }
    }
    connection.Close();
}

```

¹<https://marketplace.visualstudio.com/items?itemName=ErikEJ.SQLiteServerCompactSQLiteToolbox>



Şekil 17.1

```
/*Program Çıktısı:
(1) Ali Veli
(2) Ayşe Fatma
(3) Mehmet Can
(4) Zeynep Yılmaz
(5) Ahmet Demir
(6) Fatma Kaya
*/
```

§17.3.3 Sorgu Parametreleri

Veri tabanı için yazılacak sorgulara parametreler metin olarak yazılma yerine kapsamdaki bir değişkene bağlanabilir;

```
using Microsoft.Data.Sqlite; //Nuget Paketini Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\")); //Proje klasörü
var db = Path.Combine(projeKlasörü, "CSharp.sqlite");
var connectionString = $"Data Source={db}";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText = @" INSERT INTO Ogrenci (Adi, Soyadi) VALUES ($Ad, $Soyad)";
    string ad = "İlhan";
    string soyad = "Özkan";
    command.Parameters.AddWithValue("$Ad", ad);
    command.Parameters.AddWithValue("$Soyad", soyad);
    int rowsAffected = command.ExecuteNonQuery();
    Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
    connection.Close();
}
```

```
}
```

§17.3.4 Veritabanı İşlemleri

Veritabanı işlemlerinde (**transaction**), bazen birden fazla sorgunun birbirine bağlı olarak çalışması gerekir. Örneğin; bir banka hesabından para çekilip başka bir hesaba yatırılırken, para çekme işlemi başarılı olup yatırma işlemi başarısız olursa veri tutarsızlığı oluşur. **SqlTransaction** bu gibi durumlarda tüm işlemleri geri almamızı (**rollback**) sağlar. Veri tabanında aynı anda çok sayıda değişiklik yapıldığı durumlarda faydalıdır. İşlem akışı;

1. **BeginTransaction()**: Bağlantı üzerinden işlemi başlatır.
2. **Transaction Ataması**: Oluşturulan işlem nesnesi, ilgili komutun (**SqlCommand.Transaction**) özelliğine atanır.
3. **Commit()**: Her şey yolundaysa değişiklikleri veritabanına kalıcı olarak kaydeder.
4. **Rollback()**: Bir hata oluşursa, işlem başladığından beri yapılan tüm değişiklikleri iptal eder.

Aşağıda veri tabanında güvenli veri güncelleme örneği verilmiştir;

```
using (SqlConnection baglanti = new SqlConnection(baglantiCumlesi))
{
    baglanti.Open();
    // 1. İşlemi (Transaction) başlatıyoruz.
    SqlTransaction islem = baglanti.BeginTransaction();
    SqlCommand komut = baglanti.CreateCommand();
    komut.Transaction = islem; // Komutun bu işleme ait olduğunu belirttik.
    try
    {
        // Birinci sorgu
        komut.CommandText = "UPDATE Hesaplar SET Bakiye = Bakiye - 100 WHERE ID = 1";
        komut.ExecuteNonQuery();
        // İkinci sorgu
        komut.CommandText = "UPDATE Hesaplar SET Bakiye = Bakiye + 100 WHERE ID = 2";
        komut.ExecuteNonQuery();
        // 2. Her şey başarılıysa onayla.
        islem.Commit();
        Console.WriteLine("İşlem başarıyla tamamlandı.");
    }
    catch (Exception ex)
    {
        // 3. Hata varsa tüm adımları geri al!
        islem.Rollback();
        Console.WriteLine("Hata oluştu, değişiklikler geri alındı: " + ex.Message);
    }
}
```

Kurumsal veri tabanlarının aksine SQLite veri tabanında, aynı anda yalnızca bir işlemin beklemede olan değişikliklere sahip olmasına izin verilir. Bu durum **eş zamanlılık (concurrency)** olarak adlandırılır. Bu nedenle, **SQLiteCommand** ile başlanan işte **BeginTransaction** ve **Execute** yöntemlerine yapılan çağrılar, başka bir işlemin tamamlanması çok uzun sürerse zaman aşımına uğrayabilir². Bunu bizim SQLite veri tabanımıza Burslar adlı tabloyu ekleyerek bir öğrencinin burs miktarını değiştirme senaryosu üzerinde yapalım;

```
using Microsoft.Data.Sqlite; //Nuget Paketini Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory; //Çalıştırılabilir .NET uygulamasının
→ olduğu klasörde
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\\")); //Proje klasörü
var db = Path.Combine(projeKlasörü, "CSharp.sqlite");
var connectionString = $"Data Source={db}";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    // 1. Tabloyu Hazırlama ve Veri Ekleme
    using (var command = connection.CreateCommand())
    {
        command.CommandText = @"
```

²<https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/transactions>

```

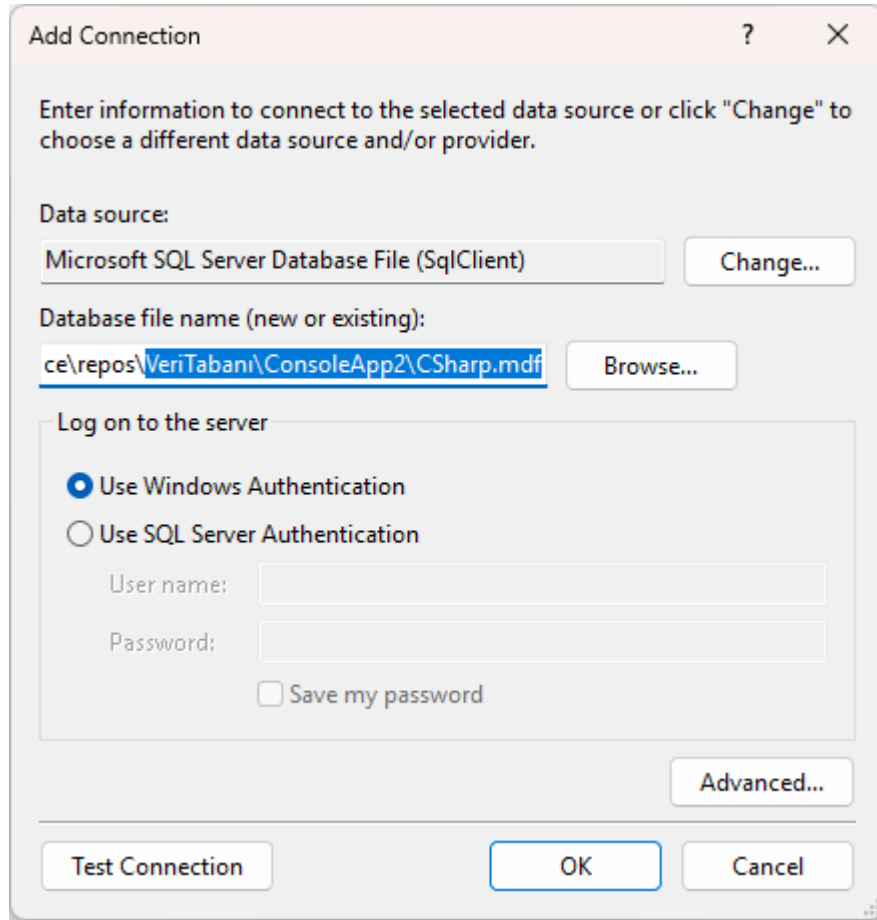
DROP TABLE IF EXISTS Burslar;
CREATE TABLE Burslar (
    OgrenciId INTEGER NOT NULL,
    BursMiktari REAL NOT NULL
);
INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (1,1000.0);
INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (2,2000.0);
INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (3,3000.0);
INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (4,4000.0);
INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (5,3000.0);
";
command.ExecuteNonQuery();
}
// 2. Transaction İşlemi: SQLite'da 'deferred' varsayılan moddur; yazma anına kadar
↪ kilitlemeyi erteler.
using (var transaction = connection.BeginTransaction())
{
    try
    {
        using (var command = connection.CreateCommand())
        {
            // Komutu başlattığımız işleme (transaction) bağlıyoruz.
            command.Transaction = transaction;
            // Örnek Senaryo: Öğrenci 1'in bursuna 500 TL ekleme yapıyoruz.
            long ogrenciId = 1;
            decimal ekBurs = 500.0m;
            command.CommandText = "UPDATE Burslar SET BursMiktari = BursMiktari + @Miktar
            ↪ WHERE OgrenciId = @Id";
            // Parametre kullanımı güvenlik için şarttır!
            command.Parameters.AddWithValue("@Miktar", ekBurs);
            command.Parameters.AddWithValue("@Id", ogrenciId);
            int etkilenenSatir = command.ExecuteNonQuery();
            if (etkilenenSatir == 0)
            {
                // Eğer güncellenecek öğrenci bulunamadıysa işlemi iptal edebiliriz.
                throw new Exception("Güncellenecek öğrenci kaydı bulunamadı!");
            }
            // 2. Her şey başarılıysa değişiklikleri kalıcı hale getir.
            transaction.Commit();
            Console.WriteLine($"{ogrenciId} ID'li öğrencinin bursu başarıyla güncellendi.");
        }
    }
    catch (Exception ex)
    {
        // 3. Hata varsa (bağlantı kopması, SQL hatası vb.) tüm adımları GERİ AL!
        transaction.Rollback();
        Console.WriteLine("İşlem başarısız, veritabanı eski haline döndürüldü: " +
        ↪ ex.Message);
    }
}
}

```

17.4 Veri Kaynaklarına Erişim Yöntemleri

.NET Framework ile veri kaynaklarına iki stratejiyle erişim yapılır;

1. Bağlı (**connected**): Bu yöntemde veri üzerindeki değişiklikler doğrudan veri kaynağı üzerinde yapılır. Veri kaynakları pahalı yatırımlarla kurulduğundan, kaynağı en az meşgul edecek işlemler için veya zorunlu durumlarda kullanılır. .NET teknolojisinde bunu **yukarıdaki örneklerde verildiği gibi DataReader** nesnesi ile yaparız.
2. Kopuk (**disconnected**): Bu yöntemde veriler bellekte geçici bir yere aktarılırlar. Veri seti olan ve **DataSet** olarak adlandırılan bu yer, sanki bir veri kaynağıymış gibi kullanılır. Veri üzerinden işlemler tamamlandığında, bellekteki değişiklikler veri kaynağına kaydedilir. Kısaca veri setleri, **hafızada oluşturulan**



Şekil 17.2: Microsoft SQL Server LocalDB Üzerinde Veri Tabanı Oluşturma

bir veri tabanlarıdır.

Veri Seti için *Microsoft SQL Server LocalDB* sürümünü kullanacağız. Bu veritabanının kurulumu, **Visual Studio** kurulumunda veri araçları seçilerek yapılabilir. Kurulum yapıldıktan sonra, yeni bir proje oluşturup **Tools** → **Connect To Database...** seçilir. Açılan pencerede yeni bir veri tabanı **proje klasöründe** oluşturulur;

Oluşan veri tabanına **View**→**Server Explorer** menüsünden ulaşılabilir. Eğer **.NET Core CLI** ile proje oluşturmuş iseniz veri tabanını yönetmek için **SQL Server Management Studio** yazılımını kullanabilirsiniz³.

```
using Microsoft.Data.SqlClient; //Nuget Paketini Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
var db = Path.Combine(projeKlasörü, "CSharp.mdf");
string connectionString = $"Server=(LocalDB)\\MSSQLLocalDB; AttachDbFilename={db}; Integrated
→ Security=True;Connect Timeout=30;";
using (var connection = new SqlConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText = @"
        DROP TABLE IF EXISTS Oğrenci;
        CREATE TABLE Oğrenci (
            OğrenciId INT PRIMARY KEY IDENTITY (1, 1),
            Adi NVARCHAR (50) NOT NULL,
            Soyadi NVARCHAR (50) NOT NULL
        );
        DROP TABLE IF EXISTS Ders;
        CREATE TABLE Ders(
            DersId INT PRIMARY KEY IDENTITY (1, 1),
            DersAdi NVARCHAR(50) NOT NULL
        );
    ";
}
```

³<https://learn.microsoft.com/en-us/ssms/install/install>

```

    );
";
int rowsAffected = command.ExecuteNonQuery();
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
command.CommandText = @"
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ali', 'Veli');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ayşe', 'Fatma');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Mehmet', 'Can');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Zeynep', 'Yılmaz');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ahmet', 'Demir');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Fatma', 'Kaya');
    INSERT INTO Ders (DersAdi) VALUES ('Matematik');
    INSERT INTO Ders (DersAdi) VALUES ('Fizik');
";
rowsAffected = command.ExecuteNonQuery();
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");

DataSet dataSet = new DataSet();
dataSet.Tables.Add("Ogrenci");
dataSet.Tables.Add("Ders");

SqlDataAdapter dataAdapterOgrenci = new SqlDataAdapter("Select * From Ogrenci", connection);
dataAdapterOgrenci.Fill(dataSet.Tables[0]); /* veriler veritabanından okunup verisetine
↳ yüklendi. veritabanına bağlatı (connection) sadece bu an için yapıldı. */

SqlDataAdapter dataAdapterDers = new SqlDataAdapter("Select * From Ders", connection);
dataAdapterDers.Fill(dataSet.Tables[1]); /* veriler veritabanından okunup verisetine
↳ yüklendi. veritabanına bağlatı (connection) sadece bu an için yapıldı. */

connection.Close(); //Bağlantı kapatıldıktan sonra veri setleriyle işlemler yapılabilir!

Console.WriteLine("OgrenciId,Adı,Soyadı");
foreach (DataRow row in dataSet.Tables[0].Rows)
    Console.WriteLine("{0},{1},{2}", row[0], row[1], row[2]);
Console.WriteLine("DersId,DersAdi");
foreach (DataRow row2 in dataSet.Tables[1].Rows)
    Console.WriteLine("{0},{1}", row2[0], row2[1]);
//Bağlantı kapatılmadan burada çeşitli tablo satırlarını (row) güncelleyebiliriz.
}

```

17.5 NoSQL Veri Tabanı

NoSQL veri tabanları ilişkisel veri tabanları kadar veri tutarlılığı sağlamaz ve verileri düzenlemek çok daha zordur. Bu nedenle genellikle çok sayıda veri yazmak ve çok sayıda veriyi okumak için kullanılır. Aşağıda bir okuldaki öğrenciler ve diplomalarının saklandığı bir NoSQL örneği verilmiştir. Diploma örneği olarak [CSharp.R3.pdf](#) dosyasını proje klasörüne koymalıyız.

```

using LiteDB; //Nuget Paketini (by Mauricio David) Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\\"));
var veritabani = Path.Combine(projeKlasörü, "CSharp.Litedb");
using (var db = new LiteDatabase(veritabani))
{
    var öğrenciler = db.GetCollection<Öğrenci>("Öğrenciler");
    var öğrenci = new Öğrenci
    {
        Adı = "İlhan Özkan",
        EPosta = "ilhanozkan@outlook.com",
        CepTelefonu = "03121234567"
    };
    // Yeni Öğrenci koleksiyona eklenir (Id otomatik artar)
    öğrenciler.Insert(öğrenci);
    var result = öğrenciler.Find(x => x.EPosta == "ilhanozkan@outlook.com").FirstOrDefault();
}

```



```

Console.WriteLine($"{result.Id}-{result.Adı}");
öğrenci.CepTelefonu = "04621234567";
öğrenciler.Update(öğrenci);
var diplomalar = db.GetCollection<Diploma>("Diplomalar");
byte[] fileBytes = File.ReadAllBytes(Path.Combine(projeKlasörü, "CSharp.R3.pdf")); //PDF
    ↳ dosyayı veritabanına
var diploma = new Diploma
{
    ÖğrenciId = öğrenci.Id,
    PdfDocument = fileBytes
};
diplomalar.Insert(diploma);
}
public class Diploma
{
    public int Id { get; set; }
    public int ÖğrenciId { get; set; }
    public byte[] PdfDocument { get; set; }
}
public class Öğrenci
{
    public int Id { get; set; }
    public string Adı { get; set; }
    public string EPosta { get; set; }
    public string CepTelefonu { get; set; }
}

```

Veri tabanına yazdırılan nesneleri okumak için aşağıdaki kod kullanılabilir;

```

using LiteDB; //Nuget Paketini (by Mauricio David) Kur!
var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\\"));
var veritabani = Path.Combine(projeKlasörü, "CSharp.litedb");

using (var db = new LiteDatabase(veritabani))
{
    var öğrenciler = db.GetCollection<Öğrenci>("Öğrenciler");
    var liste = öğrenciler.FindAll();
    foreach (var o in liste)
        Console.WriteLine($"{o.Id}-{o.Adı}-{o.EPosta}-{o.CepTelefonu}");
    var diplomalar = db.GetCollection<Diploma>("Diplomalar");
    var result = diplomalar.Find(x => x.ÖğrenciId == 1).FirstOrDefault();
    File.WriteAllBytes(Path.Combine(projeKlasörü, "CSharp.R3.Veritabanından.pdf"),
        ↳ result.PdfDocument);
    //veritabanından dosya sistemine dosyayı yazdık.
}
public class Diploma
{
    public int Id { get; set; }
    public int ÖğrenciId { get; set; }
    public byte[] PdfDocument { get; set; }
}
public class Öğrenci
{
    public int Id { get; set; }
    public string? Adı { get; set; }
    public string? EPosta { get; set; }
    public string? CepTelefonu { get; set; }
}

```

17.6 Düşün ve Çöz

- ◇ **Düşün:** İlişkisel (SQL) ve ilişkisel olmayan (NoSQL) veri tabanları arasındaki temel tasarım felsefesi farkı nedir? ACID (Atomiklik, Tutarlılık, İzolasyon, Dayanıklılık) özellikleri neden ilişkisel veri tabanlarında kritik bir öneme sahiptir?
- ◇ **Düşün:** SQL enjeksiyonu (**SQL injection**) saldırısı nedir? Kullanıcı girdisini **string** birleştirme ile sorguya eklemek neden tehlikeli olduğunu somut bir örnekle gösterin. Parametrelili sorgular bu saldırıyı nasıl engeller?
- ◇ **Düşün:** Bağlantı havuzu (**connection pooling**) neden gereklidir? Her işlemde yeni bir veri tabanı bağlantısı açmanın maliyeti nedir? **using** bloğuyla bağlantı yönetiminin bağlantı havuzuyla etkileşimi nasıldır?
- ◇ **Düşün:** İşlem (**transaction**) yönetimi neden gereklidir? Banka havalesi senaryosunda gönderen hesaptan para düşüldükten sonra alıcı hesaba para eklenemezse ne olur? **Rollback** mekanizması bu durumu nasıl çözer?
- ◇ **Düşün:** ADO.NET ile doğrudan SQL yazmak ile bir ORM kullanmak arasındaki ödünleşimler (**trade-offs**) nelerdir? Performans kritik bir raporlama ekranında neden doğrudan SQL tercih edilebilir?
- ◇ **Düşün:** SQLite, MSSQL ve PostgreSQL arasındaki seçimi etkileyen faktörler nelerdir? Gömülü uygulama, web API ve kurumsal analitik için hangisi neden tercih edilir?
- ◇ **Düşün:** N+1 sorgu problemi nedir? Bir yazarın tüm kitaplarını çeken uygulamada bu problemin nasıl oluştuğunu ve tek bir **JOIN** sorgusuyla nasıl çözüldüğünü açıklayınız.
- ◇ **Çöz:** SQLite kullanarak bir kütüphane veri tabanı oluşturun: Kitaplar ve Yazarlar tabloları arasında ilişki olsun. CRUD işlemlerinin (ekleme, listeleme, güncelleme, silme) her biri için parametrelili sorgu yazınız.
- ◇ **Çöz:** İşlem (**transaction**) kullanarak iki hesap arasında para transferi gerçekleştiren program yazınız: gönderen ve alıcı hesap bilgileri kullanıcıdan alınsın; yetersiz bakiye veya hesap bulunamama durumlarında geri alma (**rollback**) yapılsın.
- ◇ **Çöz:** SQL enjeksiyonu açığı içeren bir sorgu yazın (bilerek), ardından parametrelili sorguyla güvenli versiyonunu yazın. Her iki versiyonu **OR 1=1** girdisiyle test edin ve farkı gözlemleyin.
- ◇ **Çöz:** Öğrenci ve Ders tablolarını içeren bir veri tabanı oluşturun. LINQ benzeri bir arayüzle (yöntem zincirleme) SQL sorgusu oluşturan basit bir sorgu oluşturucu (**query builder**) sınıfı tasarlayın.
- ◇ **Çöz:** ADO.NET ile bir ürün envanteri uygulaması yazın: ürün arama (ada veya kategoriye göre), stok güncelleme ve düşük stoklu ürünleri listeleyen raporlama yöntemleri içersin.